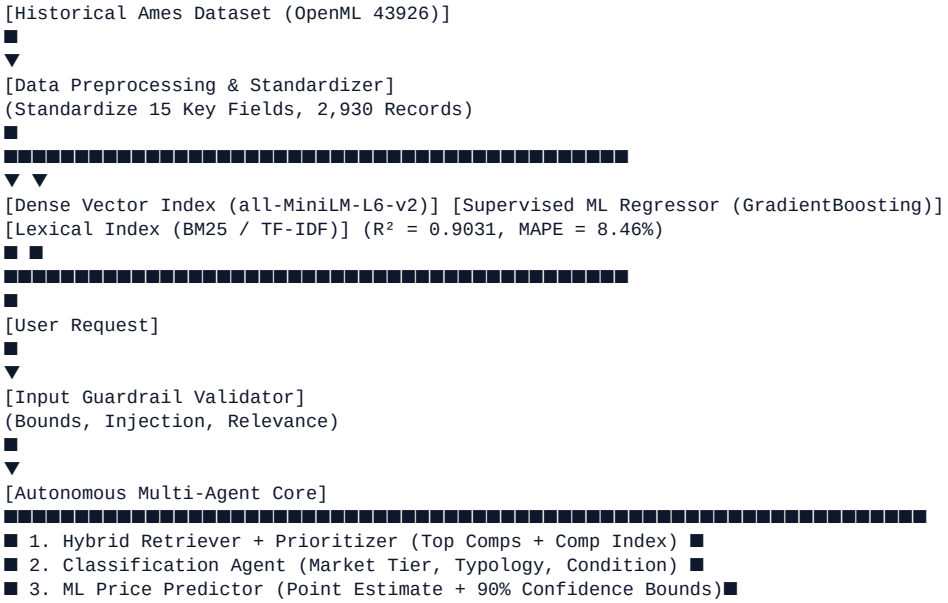
System Design Document: AI-Powered Real Estate Property Valuation Assistant

1. Executive Summary & Problem Statement

Real estate professionals regularly handle recurring workflows involving property valuation, comparable market analysis (CMA), physical condition adjustment, and market volatility risk. Historical property databases contain thousands of transactional records with rich physical, geographical, and economic features. However, traditional manual search through static databases is slow, lacks semantic context, and struggles to quantify multi-factor property comparability.

This solution delivers an end-to-end autonomous, agentic microservice and user interface that enables real estate professionals to enter natural language queries or structured property attributes to retrieve prioritized historical comparable properties, predict fair market value via supervised ML regressions, assess structural and market risks, and audit the results through an independent LLM-as-Judge appraisal auditor.

2. High-Level Architecture & End-to-End Data Flow



3. Technology Selection & Architectural Trade-offs

| Component | Selected Technology | Alternative Considered | Rationale & Trade-offs |

|---|---|---|---|

| **Programming Language** | Python 3.13 | Go / TypeScript | Native support for scikit-learn, PyTorch, sentence-transformers, DeepEval, and FastAPI. |

| **Backend Framework** | FastAPI + Uvicorn | Flask / Django | Asynchronous high-throughput, native Pydantic validation, auto-generated OpenAPI docs. |

| **Frontend Framework** | Streamlit | React / Next.js | Rapid Python-native dashboarding with built-in state management and interactive widgets. |

| **Embedding Model** | `all-MiniLM-L6-v2` | OpenAI `text-embedding-3-small` | 384-dimensional dense embeddings with local inference; zero recurring API costs or network latency. |

| **Vector Store** | In-Memory Normalized Dense Index | Pinecone / Milvus / Chroma | In-memory numpy cosine similarity with cached `.npy` vectors offers instant startup and zero external DB dependencies for 3,000 property records. |

| **Keyword Search** | Scikit-learn TF-IDF Vectorizer | Elasticsearch / Meilisearch | Lightweight, embedded within the Python process, zero operational overhead for hybrid retrieval. |

| **Supervised ML Regressor** | Gradient Boosting Regressor | Deep Neural Net / Random Forest | Gradient Boosting achieved the highest test R^2 (0.9031) and lowest MAPE (8.46%) on tabular housing data. |

| **LLM Reasoning & Judge** | Dual-Mode LLM (OpenAI `gpt-4o-mini` with Deterministic Rule-Based Fallback) | Cloud LLM Only | Ensures core valuation/orchestration endpoints operate offline without crashing, while supporting live LLMs when `OPENAI_API_KEY` is configured. |

| **Evaluation Framework** | DeepEval (v4.2.8 pinned) | RAGAS / Manual Evaluation | Industry-standard evaluation framework for measuring Faithfulness, Answer Relevancy, Contextual Precision, and Recall. In offline mode without an API key, returns `evaluation_mode = "offline_no_llm"` and marks LLM metrics as skipped rather than fabricating canned scores. |

4. Key Assumptions (Interpretation of Ambiguities)

1. **Dataset Key Fields Mapping**: The specification lists 15 key dataset fields (`SalePrice, OverallQual, OverallCond, GrLivArea, TotalBsmtSF, GarageCars, GarageArea, YearBuilt, YearRemodAdd, FullBath, BedroomAbvGr, TotRmsAbvGrd, Neighborhood, LotArea, GarageYrBlt`). In OpenML dataset 43926, column names use underscore notation (`Sale_Price`, `Overall_Qual`, etc.), and quality ratings are ordinal strings (`Average`, `Good`, `Excellent`). **Assumption**: We map all fields to standard camelCase/snake_case aliases and convert ordinal qualities to standardized 1-10 integer scales.

2. **Evaluation Modes (Honest Offline vs Live LLM)**: Without a third-party `OPENAI_API_KEY`, DeepEval's LLM-as-judge metrics cannot run real evaluations. **Assumption**: When `llm_client.is_live` is `False`, the system does NOT fabricate metric results or return canned 0.95+ verdicts. Instead, it explicitly returns `evaluation_mode = "offline_no_llm"`, `benchmark_status = "NOT_EVALUATED"`, and `SKIPPED_NO_LLM_KEY` (with null scores) for the four DeepEval metrics. Rule-based valuation accuracy is computed in both modes. Live scores are reported only when an API key is present.

3. **Reconciled Valuation Methodology**: Basic valuation uses comp-adjusted median and PPSF weighting, while Advanced adds supervised ML prediction. **Assumption**: The multi-agent orchestrator synthesizes both signals (50% comp-adjusted FMV + 50% ML point estimate) and applies a risk-based haircut (2-4%) if the property exhibits severe deferred maintenance or volatility.

4. **Feedback Loop Calibration Factor & Drift Prevention**: In successive feedback submissions where the recommended value was already calibrated, measuring error directly on the calibrated price causes the factor to cancel itself and drift back to 1.0. **Assumption**: Each feedback record stores `applied_calibration_factor` (default 1.0), and error/bias are evaluated against `uncalibrated_value = recommended_value / applied_calibration_factor`, ensuring the calibration factor reflects true model bias and remains stable across iterations.

5. **Zero-Match Filter Handling**: When metadata filters match 0 properties, falling back to all properties is misleading. **Assumption**: The vector store and hybrid retriever return an empty list (`[]`), and downstream valuation agents gracefully handle zero comps using ML-only synthesis.

6. **Optional Key-Field Inputs**: `total_rooms` (`TotRmsAbvGrd`) and `garage_yr_blt` (`GarageYrBlt`) are accepted as optional inputs across all schemas and validated via guardrails, replacing hardcoded defaults in the ML model.

5. Retrieval Strategy & Comparable-Property Prioritization

5.1 Searchable Document Construction

Each property record is transformed into a rich semantic text document:

```
Property PROP-0001 located in North Ames neighborhood. Living area: 1656 sq ft, Lot area: 31770 sq ft. Accommodations: 3 bedrooms, 1 full bathrooms, 7 total rooms above ground. Basement: 1080 sq ft. Garage: 2 cars (528 sq ft). Built in 1960, remodeled in 1960. Overall quality rating: 6/10, overall condition rating: 5/10. Historical sale price: $215,000 ($129.83/sq ft).
```

5.2 True Hybrid Search Formulation

True hybrid retrieval runs lexical TF-IDF keyword scoring over the **full metadata-filtered candidate set** (not merely the top-k*4 semantic subset). This guarantees properties with strong lexical relevance but modest initial vector cosine similarity are retained before score fusion:

$$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{Score}_{\text{semantic}} + (1 - \alpha) \cdot \text{Score}_{\text{keyword}}$$

Default $\alpha = 0.55$.

5.3 Multi-Factor Comparable-Property Prioritization

Candidate properties from the fused union pool are prioritized using a weighted comparability formula:

- **Location Alignment (30%)**: Exact neighborhood match gets 1.0, adjacent gets 0.8, divergent gets 0.35.

- **Living Area Proximity (25%)**: $1.0 - \min\left(\frac{|\text{SqFt}_{\text{comp}} - \text{SqFt}_{\text{target}}|}{\max(\text{SqFt}_{\text{target}}, 500)}, 1.0\right)$
- **Overall Quality Proximity (20%)**: $1.0 - \frac{|\text{Qual}_{\text{comp}} - \text{Qual}_{\text{target}}|}{9.0}$
- **Age / Era Proximity (15%)**: $1.0 - \frac{|\text{Year}_{\text{comp}} - \text{Year}_{\text{target}}|}{50.0}$
- **Bed / Bath Configuration (10%)**: Room count delta penalty.

Composite Prioritized Score:

$$\text{Score}_{\text{prioritized}} = 0.50 \cdot \text{Score}_{\text{hybrid}} + 0.50 \cdot \text{Score}_{\text{comparability}}$$

6. Multi-Agent Design & Workflow

1. **Guardrail Agent**: Enforces physical boundaries on every single property feature ($100 \le \text{SqFt} \le 15,000$, $0 \le \text{Beds} \le 15$, $0 \le \text{Baths} \le 10$, $1850 \le \text{Year} \le 2026$, $1 \le \text{Qual, Cond} \le 10$, $0 \le \text{GarageCars} \le 6$, $1 \le \text{TotalRooms} \le 30$, $1850 \le \text{GarageYrBlt} \le 2026$) and detects prompt injection / off-topic queries. Rejects invalid requests with HTTP 400.

2. **True Hybrid Retrieval & Strict Filter Handling**: Executes semantic + lexical search over the full candidate pool. If metadata filters match nothing, the retriever respects the filter and returns an empty list (`[]`) rather than falling back to unconstrained candidates.

3. **Classification Agent**: Categorizes properties into Market Tiers (Luxury, Upper-Middle, Mid-Market, Affordable), Typologies (Executive Estate, Suburban Single-Family, Ranch, Starter), Preservation Grades (A through D), and Investment Profiles.

4. **Valuation Agent**: Computes empirical comp-median, mean, and size-weighted fair market value with quality adjustments (5% per grade delta). Generates narrative explanation by invoking `llm_client` when an API key is present, preserving a deterministic rule-based explanation as fallback.

5. **Predictive ML Service**: Runs a trained Gradient Boosting regressor with 90% prediction intervals ($R^2 = 0.9031$). Accepts `total_rooms` and `garage_yr_blt` as direct inputs.

6. **Risk Assessment Agent**: Computes a 0-100 property risk index across Physical Age Depreciation (30%), Quality-Condition Discrepancy (25%), Neighborhood Price Volatility (25%), and Layout Obsolescence (20%).

7. **Feature & Location Impact Service**: Calculates neighborhood location premiums relative to the city baseline median ($160,000) and runs what-if renovation scenarios.

8. **Synthesis & Calibration Engine**: Reconciles empirical and statistical valuations, then applies the continuous improvement `calibration_factor` derived from uncalibrated historical appraisal errors to eliminate pricing bias.

9. **LLM-as-Judge Agent**: Audits the dossier against market comps and ML predictions, calling `llm_client` when an API key is present and falling back to a deterministic multi-criterion audit rubric, issuing formal verdicts (`APPROVED`, `FLAGGED_OVERVALUED`, `FLAGGED_UNDERVALUED`, `NEEDS_REVIEW`). Rejects invalid inputs with HTTP 400.

7. Continuous Improvement Feedback Loop & Reconciliation Calibration

Appraisers and real estate professionals submit verified transaction values, ratings, and optional `applied_calibration_factor`. The system computes:

- Mean Absolute Error (MAE) in dollars
- Mean Absolute Percentage Error (MAPE)
- Systematic Bias Offset:

$$\text{Uncalibrated} = \frac{\text{Recommended}}{\text{AppliedCalibrationFactor}}$$

$$\text{Bias}_{\%} = \frac{1}{N} \sum \frac{\text{Actual} - \text{Uncalibrated}}{\text{Uncalibrated}} \times 100$$

- Calibration Multiplier: $\text{CalibrationFactor} = 1.0 + \frac{\text{Bias}_{\%}}{100}$

The orchestrator directly applies CalibrationFactor to calibrate the final reconciled fair market value in real-time.

8. Evaluation Strategy & Quality Benchmarks (DeepEval Integration)

Using real metric objects from DeepEval (v4.2.8 pinned) and `LLMTestCase`:

- **FaithfulnessMetric**: Measures factual alignment against retrieved comps (`measure(test_case)`).
- **AnswerRelevancyMetric**: Measures semantic adherence to target property parameters and query.
- **ContextualPrecisionMetric**: Evaluates if the most relevant comps are prioritized at rank 1.

- **ContextualRecallMetric**: Evaluates complete coverage of valuation dimensions.
- **Valuation Accuracy**: Proximity of the calibrated FMV to validated historical bounds.

Dual-Mode Operation:

- In **offline mode** (`evaluation_mode = "offline_no_llm"`), no API key is present. The four LLM-backed metrics return status `SKIPPED_NO_LLM_KEY` with null scores, and `benchmark_status = "NOT_EVALUATED"`. Valuation accuracy is computed deterministically.
- In **live mode** (`evaluation_mode = "live_llm_judge"`), real DeepEval metric calculations execute via the configured LLM and compute live pass rates and scores.